

Zero Label Cropper - Complete Technical Overview

Cropper Hub | Full documentation export

Contents (sections in this document)

1. Product overview
2. Frontend stack (frameworks, libraries, tooling)
3. Backend stack (Python, packages, Docker OS deps)
4. Data stores, queues, and object storage
5. Production deployment topology
6. Key environment variables (reference)
7. REST API surface (summary)
8. Async task and download flow
9. Repository layout (high level)
10. Development and regeneration

1. Product overview

Zero Label Cropper (internally: Cropper Hub) is a SaaS-style web application for Indian e-commerce sellers who process Meesho and Flipkart shipping-label PDFs. Core capabilities: upload multi-page PDFs; sort and crop labels by delivery partner or SKU; optional premium features (suspicious-buyer marking, pincode-based splits, multi-order handling, loyalty markers, custom print text); OCR pipelines producing CSV/Excel; return-analysis jobs; per-user wallet with coin billing; dashboards and persistent crop history; admin operations for users and credits; secure downloads of cropped PDFs or ZIP bundles via presigned object-storage URLs.

2. Frontend stack

Frameworks and runtime

- React 18.3.x (react, react-dom) with JSX.
- Vite 5.4.x as bundler and dev server (@vitejs/plugin-react).
- react-router-dom 7.x: BrowserRouter, Routes, lazy(() => import(...)) code splitting.
- ES modules (package.json type: module); Node/npm for build.
- pdf-lib 1.17.x for optional client-side PDF helpers.
- firebase 12.x SDK listed as dependency (integrations as wired in app).
- Native fetch() for HTTP (no axios dependency).
- Prettier 3.x for formatting.

Application patterns

- AuthProvider + WalletProvider wrapping the router tree.
- Session: JSON in localStorage (cropperhub_session) with bearer token for API.
- import.meta.env.VITE_API_URL: production API origin.
- vite.config.js: dev proxy /api -> http://127.0.0.1:8000 (override VITE_DEV_API_PROXY).
- Optional VITE_ARTIFACT_DOWNLOAD_TIMEOUT_MS for large downloads.
- ProtectedRoute wraps authenticated shell; AdminRoute gates /admin.
- AppLayout: Header + Outlet; theme from theme/brandTheme.js.
- Large UI: WorkspaceView (crop/OCR flows, polling), MeeshoNeonView, MyDashboardPage, AdminPage.
- src/api/taskApi.js: fetchWithRetry, fetchTaskStatus, downloadTaskArtifact (?as_json=1 for R2).

Frontend hosting

- Static build: npm run build -> dist/.
- vercel.json rewrites for SPA (client-side routes).
- Typical host: Vercel with custom domain and HTTPS.

Zero Label Cropper - Complete Technical Overview

Cropper Hub | Full documentation export

3. Backend stack

Runtime and framework

- Python 3.11 (Dockerfile: python:3.11-slim).
- FastAPI 0.115+ with Uvicorn 0.32+ (ASGI).
- Pydantic models for request/response bodies.
- python-multipart for file uploads.
- Starlette CORSMiddleware (allow_origins from env; expose_headers Location, Content-Disposition).

Python packages (requirements.txt)

fastapi, uvicorn[standard], python-multipart, pymupdf (fitz), pdfplumber, pytesseract, openpyxl, Pillow, redis>=4.6,<5, psycopyg[binary], boto3.

Docker system packages

tesseract-ocr, tesseract-ocr-eng, libgl1, libgl2.0-0 (OCR and imaging native deps).

WORKDIR /app/backend; LABELHUB_DB_PATH=/app/data/labelhub.db default; CMD start.sh runs uvicorn binding PORT.

Notable backend modules

server.py (routes), auth_store.py, history_store.py, task_queue.py, worker.py, meesho_service.py, flipkart_service.py, partner_crop.py, pdf_sort_delivery.py, pdf_sort_sku.py, label_ocr_service.py, return_analysis_service.py, hybrid/storage.py, hybrid/queue.py.

4. Data stores, queues, and object storage

- SQLite: primary app DB path via LABELHUB_DB_PATH (persistent volume on Railway recommended): users, sessions, wallet, crop_jobs, crop_job_metrics, processing_tasks (including shadow rows when QUEUE_BACKEND=redis).
- PostgreSQL: optional auth path via DB_BACKEND=postgres and DATABASE_URL.
- Redis: QUEUE_BACKEND=redis; REDIS_URL; REDIS_QUEUE_NAME; task JSON keyed per task_id; LIST for queue.
- Cloudflare R2: STORAGE_BACKEND=s3; S3_BUCKET, S3_ENDPOINT_URL, S3_REGION, S3_ACCESS_KEY_ID, S3_SECRET_ACCESS_KEY; optional S3_PREFIX; boto3 presigned URLs for GET.
- Files: temp dirs on API/worker for processing; outputs uploaded to R2 in hybrid mode.

Zero Label Cropper - Complete Technical Overview

Cropper Hub | Full documentation export

5. Production deployment topology (hybrid)

- Vercel: hosts the Vite SPA (users hit HTTPS domain).
- Railway (or similar): Docker container runs FastAPI only; connects to Redis + SQLite volume + optional Neon Postgres; reads/writes R2 for artifacts.
- Managed Redis: Railway Redis plugin (REDIS_URL shared with workers).
- VPS (e.g. Hetzner): Docker Compose runs N worker containers (deploy/hybrid/docker-compose.worker.yml), QUEUE_BACKEND=redis, STORAGE_BACKEND=s3, DISABLE_EMBEDDED_WORKER=1; pulls jobs and writes to R2.
- DNS: apex/www to Vercel; API URL configured in VITE_API_URL.
- R2 bucket CORS: allow web origins for GET/HEAD on presigned downloads.

6. Key environment variables (reference)

Frontend: VITE_API_URL, optional VITE_ARTIFACT_DOWNLOAD_TIMEOUT_MS, VITE_DEV_API_PROXY (dev only).

API (Railway): PORT, CORS_ORIGINS, LABELHUB_DB_PATH, QUEUE_BACKEND, STORAGE_BACKEND, REDIS_URL, REDIS_QUEUE_NAME, S3_* bucket credentials, optional DATABASE_URL and DB_BACKEND, ADMIN_EMAILS, API_PLATFORM, task/worker tuning (DISABLE_EMBEDDED_WORKER, EMBEDDED_WORKER_CONCURRENCY, MEESHO_CLASSIFY_WORKERS, etc.).

Worker (VPS): same Redis/S3/queue vars as API where applicable; WORKER_* idle/backoff if set.

7. REST API surface (summary)

Health: GET /api/health, GET /api/ready.

Auth: POST /api/auth/signup, /login, GET /api/auth/me, POST /api/auth/logout.

Wallet: GET /api/wallet, POST /api/wallet/spend.

Crop (async): POST /api/crop/meesho/start, POST /api/crop/flipkart/start (multipart); legacy sync-style POST /api/crop/meesho, /api/crop/flipkart may exist for compatibility.

Tasks: GET /api/tasks/{task_id}, GET /api/tasks/{task_id}/download (?as_json=1 returns JSON with presigned URL).

OCR: POST /api/ocr/labels/excel/start, GET /api/ocr/labels/tasks/{task_id}, download route; POST /api/ocr/labels/excel.

Returns: POST /api/returns/analysis/start.

History: GET /api/history/jobs, GET /api/history/jobs/{job_id}, GET /api/history/customer.

Dashboard: GET /api/me/dashboard.

Admin (restricted): metrics, jobs, users, wallet credit, audits, OCR/returns admin routes, risk/OCR downloads per user (see server.py for full list).

Zero Label Cropper - Complete Technical Overview

Cropper Hub | Full documentation export

8. Async task and download flow

- 1) User submits crop -> API creates crop_job, uploads inputs to R2 if hybrid, enqueues Redis task, inserts local SQLite shadow row.
- 2) Worker dequeues, hydrates inputs from R2, processes with PyMuPDF/Tesseract pipeline, uploads output PDF/ZIP to R2, sets Redis task success with s3:// result_path.
- 3) Browser polls GET /api/tasks/{id}; API syncs Redis state into SQLite for history UI.
- 4) Download: GET .../download?as_json=1 returns { download_url }; browser fetches presigned URL (no Bearer on R2); large files use extended client timeout on blob fetch only.

9. Repository layout (high level)

src/ : React app (pages, components, api, auth, wallet, theme, hooks).
backend/ : Python FastAPI app, requirements.txt, worker.py, services, Dockerfile context.
deploy/hybrid/ : docker-compose.worker.yml, related deploy docs/examples.
scripts/ : ops and PDF generators.
Root: Dockerfile (API image), railway.toml, vercel.json, vite.config.js, package.json.

10. Development and regeneration of this PDF

Local FE: npm run dev (Vite). Local BE: uvicorn server:app --reload --port 8000 from backend/.
This file is produced by: python scripts/generate_complete_overview_pdf.py
Output filename: Zero_Label_Cropper_Complete_Overview.pdf (project root).
Dependency: pip install fpdf2